Comparaison entre un ABR et `std::map`

Implémentation manuelle, opérations équivalentes et mesures

C++ · 200 000 valeurs distinctes · trois répétitions · rapport Typst

Résumé

Ce travail compare un arbre binaire de recherche (ABR) construit manuellement avec le conteneur standard `std::map<int, int>`. Les deux structures reçoivent la même permutation reproductible de 200 000 entiers distincts. Trois opérations sont mesurées séparément : insertion, recherche puis suppression. Sur la mesure finale, l'ABR est plus rapide pour les trois opérations. Le résultat principal est surtout algorithmique : l'ABR personnel n'offre pas de garantie logarithmique dans le pire cas, contrairement à `std::map`.

1. Introduction

Un arbre binaire de recherche organise ses valeurs autour d'une règle simple : les valeurs plus petites qu'un nœud vont dans le sous-arbre gauche et les valeurs plus grandes dans le sous-arbre droit. Cette propriété permet de suivre un seul chemin lors d'une recherche.

Le C++ fournit déjà une structure ordonnée prête à l'emploi : `std::map`. Elle associe une clé à une valeur et maintient les clés dans l'ordre. L'objectif est de comparer, sur les mêmes données et avec les mêmes opérations :

- l'ABR dont les nœuds, les pointeurs et la mémoire sont gérés dans le fichier `bst.cpp` ;
- `std::map<int, int>`, utilisé comme référence de la bibliothèque standard.

Le mot « sans bibliothèque » concerne l'ABR personnel : aucun arbre fourni par une bibliothèque n'est utilisé pour son implémentation. `std::map` est explicitement le second objet de comparaison.

2. Présentation des deux structures

2.1. ABR personnel

Un nœud contient une valeur et deux pointeurs. L'insertion, la recherche et la suppression parcourent l'arbre par comparaisons. Les doublons sont refusés.

```
struct Node {
    int value;
    Node* left;
    Node* right;
};
```

L'insertion est itérative :

```
Node* current = root_;
while (true) {
    if (value == current->value)
        return false;
    if (value < current->value) {
        if (current->left == nullptr) {
            current->left = new Node(value);
            return true;
        }
        current = current->left;
    } else {
        if (current->right == nullptr) {
            current->right = new Node(value);
            return true;
```

```
        }
        current = current->right;
    }
}
```

La suppression traite les trois cas classiques : feuille, un seul fils, ou deux fils. Avec deux fils, la valeur est remplacée par le successeur, c'est-à- dire le plus petit nœud du sous-arbre droit, puis ce successeur est supprimé.

2.2. `std::map`

Dans cette expérience, la clé et la valeur sont identiques :

```
std::map<int, int> dictionary;
dictionary.emplace(value, value);
bool present = dictionary.find(value) != dictionary.end();
dictionary.erase(value);
```

La bibliothèque prend en charge les pointeurs, les allocations et la réorganisation interne. Le standard garantit une complexité logarithmique pour la recherche, l'insertion et la suppression d'un élément.

3. Équivalence fonctionnelle

Les deux structures sont initialisées avec la séquence :

`50, 30, 70, 20, 40, 60, 80`

Le parcours infixe de l'ABR donne `20, 30, 40, 50, 60, 70, 80`, exactement comme l'itération sur les clés du `std::map`. Les recherches de 40 réussissent et celles de 45 échouent dans les deux structures. Après suppression de 30, les valeurs restantes sont `20, 40, 50, 60, 70, 80`.

Les tests exécutés dans le programme vérifient aussi :

- l'arbre vide et la première insertion ;
- les fils gauche et droit ;
- le rejet d'un doublon ;
- une suppression de feuille ;
- une suppression avec un seul fils ;
- une suppression avec deux fils ;
- le parcours infixe trié.

Sortie de validation :

```
OK arbre vide
OK racine
OK fils gauche et droit
OK feuilles
OK doublon ignore
OK recherche
OK parcours infixe
OK suppressions feuille et deux fils
OK valeurs supprimees
OK taille apres suppressions
OK hauteur croissante
OK comparaisons croissantes
OK suppression avec un fils
```

4. Méthode expérimentale

Le banc de test suit exactement le protocole suivant :

1. créer les entiers distincts `1..200_000` ;
2. les mélanger avec `std::mt19937` et la graine `2026` ;
3. mesurer l'insertion de toutes les valeurs ;
4. rechercher successivement toutes les valeurs ;

5. supprimer toutes les valeurs dans le même ordre.

Chaque structure est reconstruite pour chaque répétition. Les valeurs sont générées avant le chronométrage. Le temps d'une opération comprend donc les appels à la structure et ses allocations internes, mais pas la génération ni le mélange des données. Le chronomètre est `std::chrono::steady_clock`.

Trois répétitions sont réalisées et la moyenne est affichée en microsecondes. Le compteur de contrôle vaut `1_200_000` pour chaque structure :

> `3` répétitions × (`200 000` recherches réussies + `200 000` suppressions réussies) `= 1 200 000`

Ce contrôle confirme que les deux structures ont retrouvé et supprimé tous les éléments attendus.

5. Code du banc de test

La boucle appliquée à l'ABR est équivalente à celle appliquée à `std::map` :

```
for (int value : values)
    abr.insert(value);
for (int value : values)
    control += abr.contains(value);
for (int value : values)
    control += abr.erase(value);
```

Pour la référence standard :

```
for (int value : values)
    dictionary.emplace(value, value);
for (int value : values)
    control += dictionary.find(value) != dictionary.end();
for (int value : values)
    control += dictionary.erase(value);
```

6. Résultats de performance

Machine de mesure : Intel Core i7-8665U à 1,90 GHz, 8 processeurs logiques, Ubuntu Linux x86_64. Compilation : `g++ -O2 -Wall -Wextra -pedantic`. La commande finale a été fixée au processeur logique 0 avec `taskset -c 0`.

Les valeurs sont les moyennes de trois exécutions indépendantes. Les temps sont en microsecondes (µs).

Opération	ABR personnel	`std::map`	Plus rapide
Insertion de 200 000	49 235,983	51 347,795	ABR
Recherche de 200 000	23 182,618	48 669,277	ABR
Suppression de 200 000	36 118,331	56 963,308	ABR

Dans cette mesure, l'ABR est environ 1,04 fois plus rapide que `std::map` pour l'insertion, 2,10 fois plus rapide pour la recherche et 1,58 fois plus rapide pour la suppression. Ces ratios ne sont pas universels : ils dépendent du compilateur, de la machine, de l'allocateur et de la charge du système.

La hauteur observée de l'ABR après l'insertion est 42. L'ordre mélangé produit donc ici un arbre raisonnablement court. Cela explique pourquoi l'ABR ne subit pas le cas dégénéré pendant cette expérience.

7. Complexité théorique

Soit `h` la hauteur de l'ABR.

Opération	ABR personnel	`std::map`
Insertion	O(h), moyenne O(log n), pire O(n)	O(log n)
Recherche	O(h), moyenne O(log n), pire O(n)	O(log n)
Suppression	O(h), moyenne O(log n), pire O(n)	O(log n)
Parcours complet	O(n)	O(n)
Mémoire	O(n)	O(n)

L'ABR personnel n'est pas automatiquement équilibré. Avec une suite croissante, il devient une chaîne de hauteur `n` et le coût total de `n` insertions devient $O(n^2)$. Avec des données mélangées, sa hauteur est souvent de l'ordre de `log n`, mais cette propriété n'est pas garantie.

Le standard garantit les opérations logarithmiques de `std::map`. La structure interne concrète est masquée, mais elle est généralement fondée sur un arbre équilibré. Cette garantie est la différence algorithmique essentielle entre les deux solutions.

8. Avantages et limites

ABR personnel	`std::map`
Montre directement les nœuds, les pointeurs et les parcours.	Interface standard et réutilisable.
Permet d'adapter la structure à un besoin pédagogique ou particulier.	Pas de gestion manuelle de la mémoire.
Risque de dégénérescence si l'ordre d'insertion est défavorable.	Complexité logarithmique garantie pour les opérations principales.
Code et maintenance à la charge du programmeur.	Structure interne et détails d'équilibrage masqués.

Le résultat expérimental ne signifie pas que l'ABR personnel est toujours meilleur ou toujours moins bon. Il compare une implémentation pédagogique à un conteneur général, sur une machine et un ordre précis. En pratique, `std::map` est généralement le choix sûr lorsqu'une structure ordonnée fiable est nécessaire. L'ABR reste utile pour comprendre l'algorithme et pour les cas où un comportement ou un parcours personnalisé est recherché.

9. Conclusion

Les deux structures réalisent correctement l'insertion, la recherche et la suppression des mêmes clés. Sur 200 000 valeurs mélangées, l'ABR obtient les trois meilleurs temps dans cette exécution. Ce résultat de temps ne supprime toutefois pas l'avantage théorique de `std::map` : sa complexité logarithmique est garantie, alors que celle de l'ABR dépend de sa forme.

La conclusion la plus importante concerne la garantie de complexité. L'ABR personnel dépend de sa hauteur et peut devenir linéaire. `std::map` garantit une complexité logarithmique pour les opérations principales. L'ABR explique donc le fonctionnement interne d'une structure ordonnée, alors que `std::map` offre une solution standard et robuste pour un programme C++ réel.

Source et mesures : `03-arbre-binaire-recherche/bst.cpp` · graine 2026